



Universidad
Rafael Landívar

Identidad Jesuita en Guatemala

Universidad Rafael Landívar

Facultad de ingeniería

Manejo e implementación de archivos

Sección: 03

Ing. Luis Enrique Rojas

Facultad de Ingeniería

Facultad de Informática y sistemas

Manejo e Implementación de Archivos

Miércoles, 30 de septiembre de 2026



LABORATORIO 13
API SWAGGER & POSTMAN

Shayla Mayré De León Monterroso

Carné: 1342925

Guatemala de la Asunción, 30 de septiembre de 2026



Nombre: Shayla Mayré De León Monterroso **Carné:** 1342925

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?
R// Es una forma de comunicación entre aplicaciones por medio de internet. Permite que un programa solicite, envíe, modifique o elimine información utilizando métodos HTTP como GET, POST, PUT y DELETE.
2. ¿Qué es un endpoint?
R// Es una dirección específica dentro de una API que permite acceder a un recurso o realizar una operación determinada.
3. ¿Qué diferencia existe entre una URL y un endpoint?
R// Una URL es una dirección utilizada para localizar un recurso en internet. Un endpoint es una URL específica de una API que está preparada para recibir una solicitud y ejecutar una operación.
4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?
R// Get: consulta u obtiene información, Post: crea un nuevo recurso, Patch: modifica solamente algunos datos de un recurso, Delete: elimina un recurso.
5. ¿Qué es JSON y por qué es común en las APIs?
R// Es un formato utilizado para representar e intercambiar información mediante pares de clave y valor. Es común en las APIs porque es sencillo de leer y puede ser interpretado fácilmente por diferentes lenguajes de programación.
6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.
R// Un código de estado HTTP indica el resultado de una solicitud realizada a un servidor.
200: la solicitud se realizó correctamente
404: el recurso solicitado no fue encontrado
500: ocurrió un error interno en el servidor
7. ¿Qué información se puede encontrar en la respuesta de una API?
R// Puede incluir un código HTTP, encabezados y un cuerpo con información. Normalmente el cuerpo contiene los datos solicitados en formato JSON.
8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?
R// Postman permite enviar solicitudes directamente a una API sin necesidad de crear primero un programa. Esto facilita verificar que los endpoints, parámetros, cuerpos JSON y respuestas funcionen correctamente antes de integrarlos en una aplicación.
9. ¿Qué es y para qué sirve Swagger UI?
R// Es una herramienta que muestra de forma gráfica la documentación de una API. Permite conocer sus endpoints, métodos HTTP, parámetros y estructuras de datos.
10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?
R// Postman permite crear, organizar y ejecutar solicitudes HTTP hacia una API. Puede utilizarse para probar operaciones como GET, POST, PUT y DELETE, analizar las respuestas y verificar si la API funciona correctamente.

1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.



2. Exploración inicial

Ingresa a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.
- Indique el método HTTP utilizado. R// GET
- Indique qué parámetro necesita la solicitud. R// petId
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.
- Explique con sus palabras qué información contiene la respuesta.
R//La API devolvió correctamente la información de la mascota asociada al ID 10. La respuesta se mostró en formato JSON e incluyó datos como el identificador, nombre y otros atributos del recurso. En este caso, la mascota consultada tiene el nombre Rufus.

Evidencia: captura de Swagger mostrando la operación y su respuesta.

The screenshot displays the Swagger UI for the GET /pet/{petId} endpoint. The method is GET, and the description is 'Find pet by ID.' Below the description, it states 'Returns a single pet.' The parameters section shows a required path parameter 'petId' of type integer(\$int64) with the value 10. An 'Execute' button is visible. The responses section shows a 200 status code with a JSON response body: {"id": 10, "name": "Rufus", "photoUrls": ["example.com"]}

Name	Description
petId * required integer(\$int64) (path)	ID of pet to return 10

Execute

Responses

Curl

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/10' \
  -H 'accept: application/json'
```

Request URL

```
https://petstore3.swagger.io/api/v3/pet/10
```

Server response

Code	Details
200	Response body { "id": 10, "name": "Rufus", "photoUrls": ["example.com"] }

3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id
name
status
category
tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET /pet/{petId}.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.



POST /pet Add a new pet to the store.

Curl

```
curl -X 'POST' \
'https://petstore3.swagger.io/api/v3/pet' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "id": 13429,
  "name": "Toty",
  "category": {
    "id": 1,
    "name": "Chihuahua"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 1,
      "name": "pequeña"
    }
  ],
  "status": "disponible"
}'
```

Request URL

```
https://petstore3.swagger.io/api/v3/pet
```

Server response

Code	Details
200	Response body<pre>{ "id": 13429, "category": { "id": 1, "name": "Chihuahua" }, "name": "Toty", "photoUrls": ["string"], "tags": [{ "id": 1, "name": "pequeña" }], "status": "disponible" }</pre> Download

4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué? R// Se utilizó el método PUT, porque permite actualizar la información de un recurso existente dentro de la API.
- ¿Qué datos fueron enviados en el body? R//Se envió un JSON con la información de la mascota. Se mantuvo el ID 13429 y se modificaron principalmente el nombre a Mosho y el estado a Agotado.
- ¿Qué código HTTP devolvió la API? R// La API devolvió el código 200, lo cual indica que la solicitud fue procesada correctamente.
- ¿Cómo comprobó que el cambio realmente ocurrió? R// Se comprobó revisando el Response body, donde aparecieron los nuevos valores enviados: el nombre Mosho y el estado Agotado.



PUT **/pet** Update an existing pet.

Curl

```
curl -X 'PUT' \
  'https://petstore3.swagger.io/api/v3/pet' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 13429,
    "name": "Mosho",
    "category": {
      "id": 1,
      "name": "Chihuahua"
    },
    "photoUrls": [
      "string"
    ],
    "tags": [
      {
        "id": 1,
        "name": "pequeño"
      }
    ],
    "status": "Agotado"
  }'
```

Request URL

`https://petstore3.swagger.io/api/v3/pet`

Server response

Code	Details
200	Response body <pre>{ "id": 13429, "category": { "id": 1, "name": "Chihuahua" }, "name": "Mosho", "photoUrls": ["string"], "tags": [{ "id": 1, "name": "pequeño" }], "status": "Agotado" }</pre> Download

5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- Identifique el parámetro utilizado. R// status
- Explique dónde se envía ese parámetro. R// El parámetro se envía dentro de la URL como un parámetro de consulta. En este caso se observa de la siguiente manera: /pet/findByStatus?status=available
- Observe la estructura de la respuesta. R// La API devuelve una lista de mascotas en formato JSON. Cada mascota puede incluir información como id, category, name, photoUrls, tags y status.
- Indique qué sucede si cambia el valor del parámetro por pending. R// Al cambiar el valor de available a pending, la API devuelve únicamente las mascotas que tienen el estado pending. Esto demuestra que el parámetro permite filtrar los resultados según el estado indicado.

GET

/pet/findByStatus Finds Pets by status.



Curl

```
curl -X 'GET' \
'https://petstore3.swagger.io/api/v3/pet/findByStatus?status=available' \
-H 'accept: application/json'
```

Request URL

```
https://petstore3.swagger.io/api/v3/pet/findByStatus?status=available
```

Server response

Code	Details
200	Response body <pre>{ "id": 18, "name": "perdida" }, { "status": "available" }, { "id": 13429, "category": { "id": 1, "name": "Chihuahua" }, "name": "Toty", "photoUrls": ["string"], "tags": [{ "id": 1, "name": "pequeña" }], "status": "available" }, { "id": 1, "name": "Pet1",</pre> Download

6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID



MIA - Semana13- GET Consultar Ma POST Crear mascota + No Environment

MIA - Semana13- API > Consultar Mascota por ID Save Share

GET https://petstore3.swagger.io/api/v3/pet/13429 Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Headers 6 hidden

Key	Value	Bulk Edit Presets ...
✓ Accept	application/json	
Key	Value	Description

Body Cookies Headers 9 Test Results 200

{ } JSON Preview Visualize

```
2  "id": 13429,  
3  "category": {  
4    "id": 3,  
5    "name": "Chihuahua"  
6  },  
7  "name": "Doki",  
8  "photoUrls": [  
9    "string"  
10 ],  
11 "tags": [  
12   {  
13     "id": 1,  
14     "name": "pequeño"  
15   }  
16 ],  
17 "status": "available"  
18 }
```

- POST – Crear mascota



```
POST https://petstore3.swagger.io/api/v3/pet

{
  "name": "Doki",
  "category": {
    "id": 3,
    "name": "Chihuahua"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 1,
      "name": "pequeño"
    }
  ],
  "status": "available"
}
```

```
{
  "id": 13429,
  "category": {
    "id": 3,
    "name": "Chihuahua"
  },
  "name": "Doki",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 1,
      "name": "pequeño"
    }
  ],
  "status": "available"
}
```

- PUT – Modificar mascota



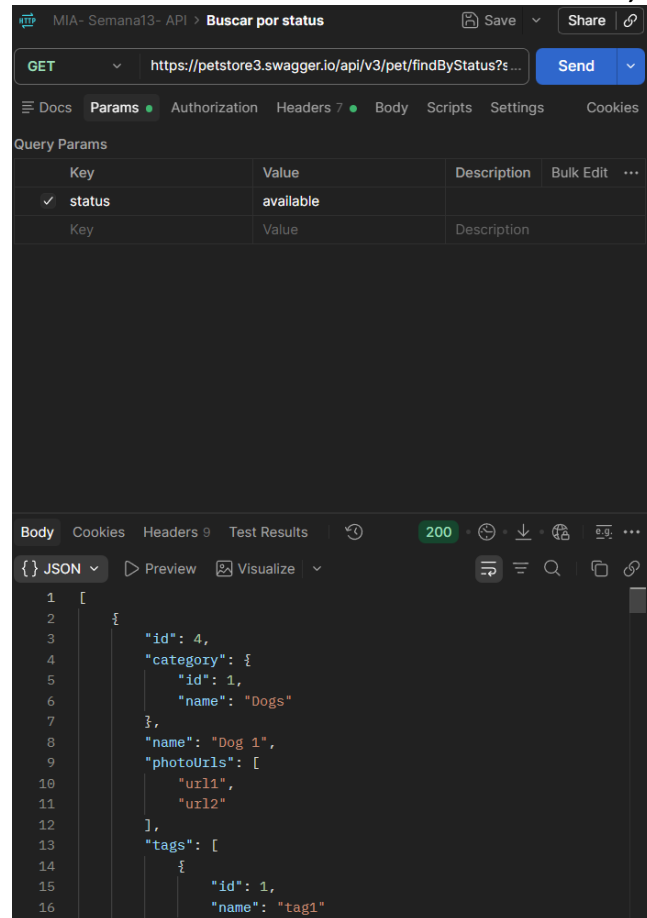
The screenshot displays the Swagger UI interface for a REST API. The top bar shows the API name 'MIA- Semana13- API' and the selected method 'PUT Modificar'. The URL is 'https://petstore3.swagger.io/api/v3/pet'. The request body is a JSON object representing a pet. The response status is 200, and the response body is also a JSON object.

```
PUT https://petstore3.swagger.io/api/v3/pet

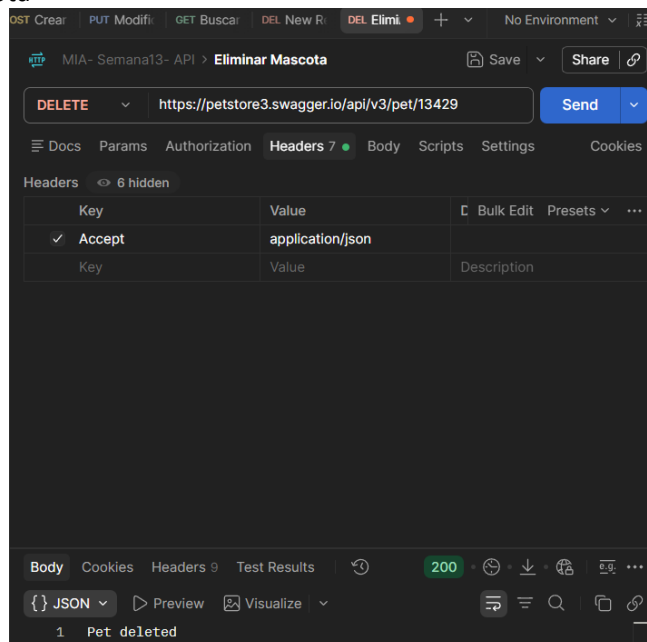
{
  "name": "Mosho",
  "category": {
    "id": 2,
    "name": "Chihuahua"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 1,
      "name": "pequeño"
    }
  ],
  "status": "Pending"
}
```

```
{
  "id": 13429,
  "category": {
    "id": 2,
    "name": "Chihuahua"
  },
  "name": "Mosho",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 1,
      "name": "pequeño"
    }
  ],
  "status": "Pending"
}
```

- GET – Buscar por status



- DELETE – Eliminar mascota



- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.

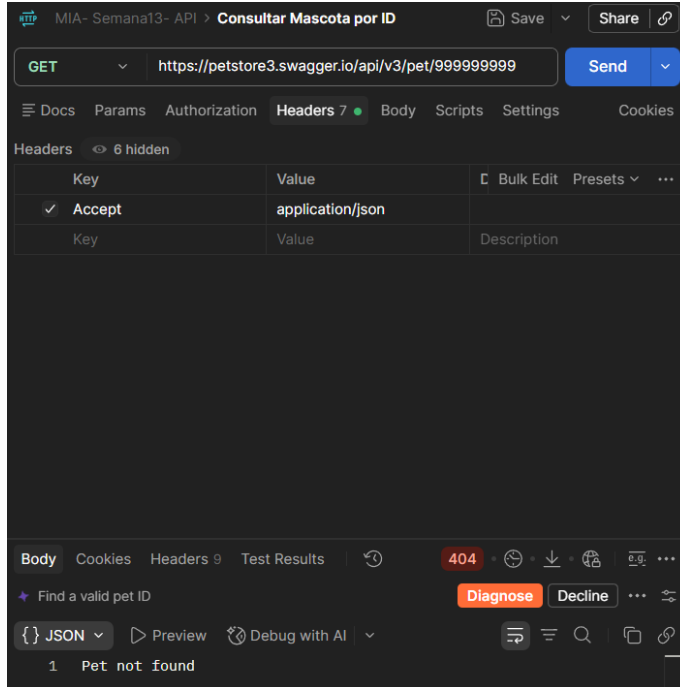
7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:



1. Consultar un ID inexistente.



a) Solicitud realizada:

Se realizó una solicitud GET al endpoint /pet/{petId} utilizando el ID 999999999.

b) Código HTTP recibido: 404 Not Found

c) Respuesta de la API:

La API devolvió el mensaje: Pet not found

d) ¿Qué esperaba que ocurriera?

Se esperaba que la API indicara que no existía ninguna mascota registrada con el ID consultado.

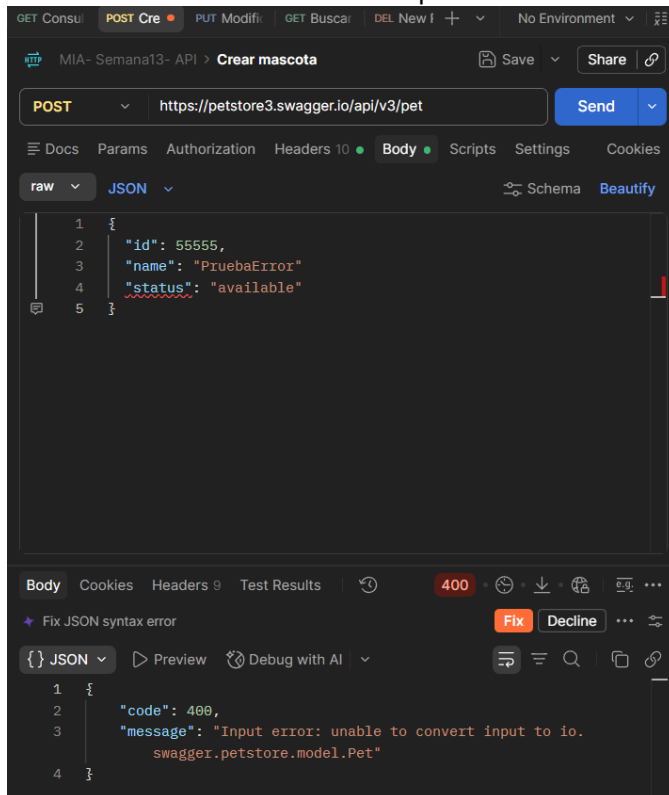
e) ¿Qué ocurrió realmente?

La API respondió con el código 404 y mostró el mensaje Pet not found, indicando que no encontró ningún recurso asociado al ID ingresado.

f) Explicación de la causa:

El error se produjo porque el ID 999999999 no corresponde a una mascota existente dentro de la API. Por esta razón, el servidor no pudo encontrar el recurso solicitado y devolvió un error 404.

2. Enviar un JSON incorrecto o incompleto.



a) Solicitud realizada:

Se realizó una solicitud POST al endpoint /pet enviando un JSON con un error de sintaxis.

b) Código HTTP recibido: 400 Bad Request

c) Respuesta de la API:

La API devolvió un mensaje indicando que no pudo convertir correctamente la entrada recibida al modelo esperado.

d) ¿Qué esperaba que ocurriera?

Se esperaba que la API rechazara la solicitud debido a que el JSON contenía un error de sintaxis.

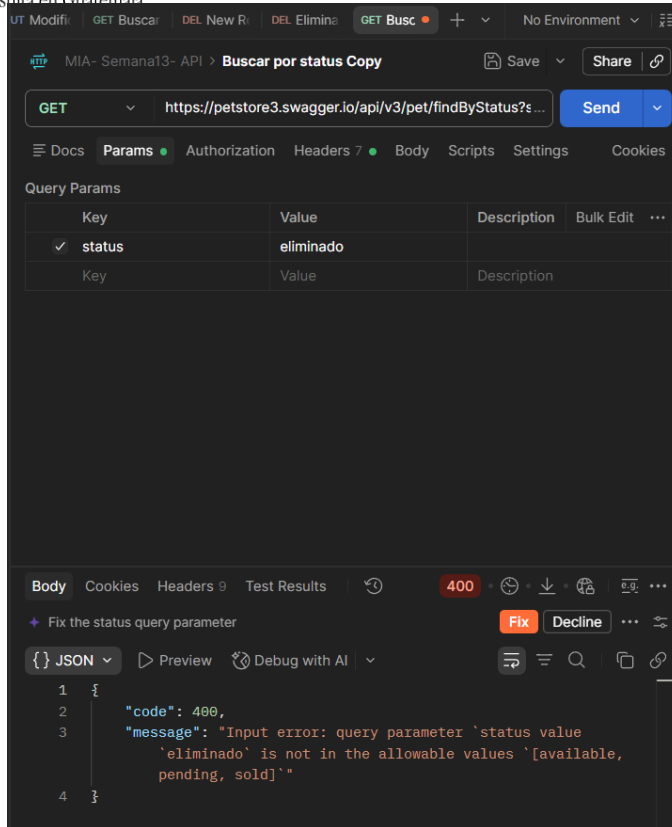
e) ¿Qué ocurrió realmente?

La API rechazó la solicitud y devolvió el código HTTP 400.

f) Explicación de la causa:

El error ocurrió porque el JSON estaba mal estructurado. Faltaba una coma entre los campos name y status, por lo que la API no pudo interpretar correctamente el contenido enviado.

3. Utilizar un parámetro con un valor no esperado.



Para cada prueba registre:

- Solicitud realizada.
- Código HTTP recibido.
- Respuesta de la API.
- Qué esperaba que ocurriera.
- Qué ocurrió realmente.
- Explicación de la causa

8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

Capture evidencia de cada paso.

Operación	Método HTTP	Endpoint	Respuesta a analizar
Registrar mascota	POST	/pet	Código HTTP y JSON de la mascota creada
Consultar mascota	GET	/pet/{petId}	Código HTTP y datos de la mascota
Actualizar mascota	PUT	/pet	Código HTTP y JSON actualizado
Eliminar mascota	DELETE	/pet/{petId}	Código HTTP de la operación
Verificar inexistencia	GET	/pet/{petId}	Código 404 y mensaje Pet not found

a) Solicitud realizada:

Se realizó una solicitud GET al endpoint /pet/findByStatus utilizando el parámetro status con el valor eliminado.

b) Código HTTP recibido: 400 Bad Request

c) Respuesta de la API:

La API indicó que el valor eliminado no forma parte de los valores permitidos para el parámetro status.

d) ¿Qué esperaba que ocurriera?

Se esperaba que la API rechazara la solicitud debido a que se utilizó un valor no válido.

e) ¿Qué ocurrió realmente?

La API rechazó la solicitud y devolvió un código 400, indicando que el parámetro tenía un valor incorrecto.

f) Explicación de la causa:

El error ocurrió porque el endpoint solo acepta determinados valores para status, específicamente available, pending y sold. El valor eliminado no está permitido.



1. Registrar mascota

Se registró una mascota mediante el método POST utilizando el endpoint /pet. El programa debe analizar el código HTTP recibido y el cuerpo de la respuesta para confirmar que el recurso fue creado correctamente.

The screenshot shows a REST client interface with a POST request to `https://petstore3.swagger.io/api/v3/pet`. The request body is a JSON object representing a pet. The response is a 200 status code with a JSON body containing the created pet's details.

```
POST https://petstore3.swagger.io/api/v3/pet

{
  "id": 13429,
  "name": "Toty",
  "category": {
    "id": 1,
    "name": "Chihuahua"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 1,
      "name": "pequeña"
    }
  ],
  "status": "available"
}
```

2. Consultar la mascota

Se consultó la mascota utilizando el método GET y su ID. El programa debe analizar el código HTTP y la información recibida para comprobar que el recurso existe.



GET <https://petstore3.swagger.io/api/v3/pet/13429> Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 9 Test Results 200

{ } JSON Preview Visualize

```
2  "id": 13429,  
3  "category": {  
4    "id": 1,  
5    "name": "Chihuahua"  
6  },  
7  "name": "Toby",  
8  "photoUrls": [  
9    "string"  
10 ],  
11 "tags": [  
12   {  
13     "id": 1,  
14     "name": "pequeña"  
15   }  
16 ],  
17 "status": "available"  
18 }
```

3. Actualizar su estado
Se actualizó el estado de la mascota mediante el método PUT. El programa debe verificar el código HTTP y revisar el cuerpo de la respuesta para comprobar que el nuevo estado fue aplicado.



```
PUT https://petstore3.swagger.io/api/v3/pet

raw JSON Schema

1 {
2   "id": 13429,
3   "name": "Mosho",
4   "category": {
5     "id": 2,
6     "name": "Chihuahua"
7   },
8   "photoUrls": [
9     "string"
10  ],
11  "tags": [
12    {
13      "id": 1,
14      "name": "pequeño"
15    }
16  ],
17  "status": "Sold"
18 }
```

4. Eliminar la mascota

Se eliminó la mascota utilizando el método DELETE y su identificador. El programa debe analizar el código HTTP recibido para determinar si la eliminación se realizó correctamente.

```
DELETE https://petstore3.swagger.io/api/v3/pet/13429

Headers 7 Body Scripts Settings

Headers 6 hidden

Key Value Bulk Edit
✓ Accept application/json
Key Value Description

Body Cookies Headers 9 Test Results

200

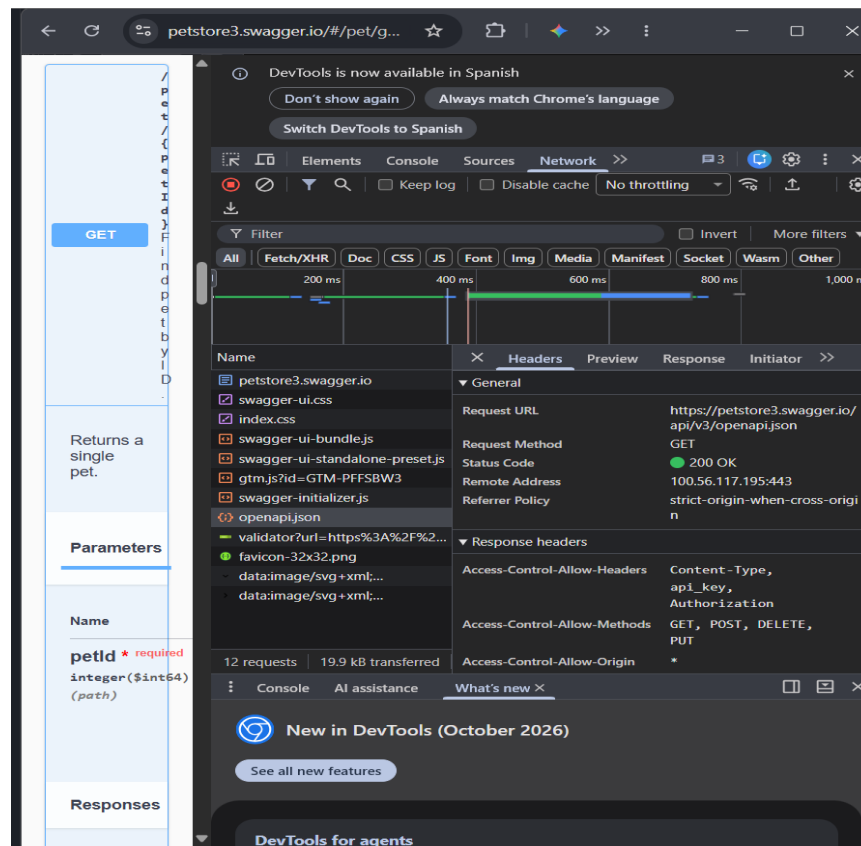
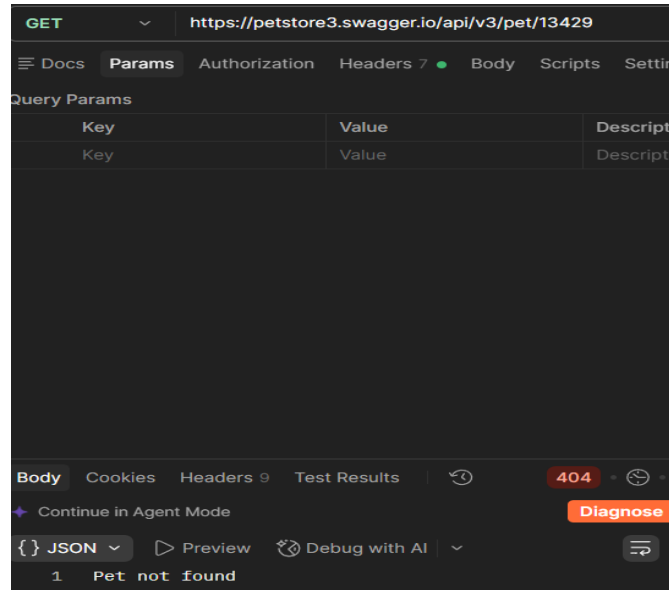
{} JSON Preview Visualize

1 Pet deleted
```



5. Comprobar que ya no existe

Finalmente, se realizó nuevamente una consulta GET utilizando el mismo ID. Si la mascota ya no existe, el programa debe identificarlo mediante el código HTTP 404 y el mensaje de respuesta Pet not found. De esta forma, el sistema puede diferenciar entre una consulta exitosa y un recurso inexistente.



Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las



Universidad
Rafael Landívar

Identidad Jesuita en Guatemala

pruebas realizadas.

Enviar link de github a plataforma de la universidad.

Facultad de Ingeniería

Facultad de Informática y sistemas

Manejo e Implementación de Archivos

Miércoles, 30 de septiembre de 2026